

从零构建 Agent

一份可照着做的实操手册。读者 = 你自己,目标 = 从零搭一个能跑、可验证、不失控的 agent。每一层都给:目录结构 / 模板 / 代码骨架 / 决策清单。



BUILD · OPERATE · SCALE

10 章 · 2026

■ 目录 · CONTENTS

- | | |
|-------------------------------------|--------------------------------|
| 00 三句话先建立坐标系 | 01 规范驱动开发(SDD)——一切的地基 |
| 02 工具层:MCP —— 一次集成,处处可用 | 03 能力层:Skills —— 教 agent "怎么做" |
| 04 协作层:A2A —— 单 agent 扛不住时才拆 | 05 护栏与零信任 —— 给永不睡觉的工具装边界 |
| 06 Loop 层 —— 让 agent 不靠你一拍一拍地敲 | 07 质量评估与过程审计 —— 那个"能说不"的东西 |
| 08 记忆与会话 —— agent 记不记得住 | 09 落地路径:从零到一条可执行的路 |
| 10 从原型到生产与规模化 —— 让一个 agent 扛住很多人同时用 | |

00 三句话先建立坐标系

- 1. **Agent = Model + Harness**。Model 是大脑(推理),Harness 是手脚和工作台(工具、记忆、循环逻辑)。你写的几乎全是 Harness。
- 2. **你的产出不是代码,而是"生产代码的工厂"**。角色从程序员 → 技术架构师。代码是**可丢弃的**;只要规范扎实,整个代码库可以反复重生成。
- 3. **瓶颈已经从"写代码"转移到下游的人**。所以这份手册一半篇幅在讲怎么让产出**可验证、可集成、不失控**,而不是怎么让 AI 写得更快。

一个判断贯穿全书:**vibe coding(让 AI 快速生成原型)是好的**;**vibe in production(把未验证的 AI 输出直接上生产)是危险的**。

◇ 全书地图

层	解决什么	关键产物
1 规范驱动 SDD	让 AI 不用猜	specs/ 文件夹
2 工具层 MCP	让 agent 够得着外部系统	MCP server / client
3 能力层 Skills	教 agent "怎么做"一类活	SKILL.md
4 协作层 A2A	单 agent 扛不住时拆	sub-agent 流水线
5 护栏 / 零信任	防止有用工具变成不可预测工具	沙箱 / HITL / Policy
6 Loop 层	让 agent 不靠你一拍一拍地敲、自己跑	automation / 状态文件
7 评估与审计	判断它干得好不好、怎么干的、能否追溯	rubric / trace / 决策日志
8 记忆与会话	agent 记不记得住、会话内外的状态	context 窗口 / STATE.md / RAG
9 落地路径	把上面串成一条可执行的路	checklist
10 生产与规模化	真正上线、运维、扛住很多人同时用	AgentOps / 高并发架构

注 · NOTE

前 5 层武装 agent 的**单次运行**(harness 及以下);第 6 层 Loop 让它**自动重来**;第 7 层评估/审计是贯穿的"能说不";第 8 层记忆/会话是栈里的 **context 层**。前八层都在讲**怎么造(BUILD)**;第 10 层是另一半**——造好之后怎么交付、运维、扛并发(OPERATE & SCALE)**。这条 prompt → context → harness → loop 的栈,是 2026 年起被称作 **Loop Engineering** 的思路(Addy Osmani 命名,引自 Peter Steinberger、Boris Cherny)。

01 规范驱动开发(SDD)——一切的地基

核心动作:把"代码优先"改成"规范优先"。有个模糊想法就开敲编辑器,是旧习惯。新习惯是先把规范写扎实——它是"架构北极星",防止 AI 看着过时的文件快照越跑越偏(上下文碎片化,像传声筒游戏)。

注 · NOTE

实操贴士:先在普通文档里写技术设计,让真人(或你自己冷静)评审一遍,宁可现在发现逻辑缺陷,也别等 AI 已经生成几千行废码,再导出成 Markdown 放进 [specs/](#)。

1.1 一份好规范包含的四件东西

1. **完整技术设计** —— 不要只说"做个登录页",拆到需求、数据库 schema、API 合约。
2. **视觉辅助** —— 图表 + 带版本号的工具/库清单。
3. **背景信息("为什么")** —— 告诉 agent 意图,它才能有前瞻性,而不是字面执行。
4. **场景与边缘情况** —— 什么是对的、什么是错的、出错怎么办。

1.2 用什么格式:Markdown + 条件式 YAML

LLM 对指令格式很敏感。经验法则:

- › **默认 Markdown**(叙述性指令)—— 标题用来锚定模型注意力。
- › **嵌套深度 > 3 的结构化配置切到 YAML** —— 深层嵌套时 YAML 比 JSON/XML 更省 token 也更易解析。

注 · NOTE

硬道理:**token 是物理约束**。每个字符、换行、缩进都换算成预算和延迟。把 [specs/](#) 当成"精简编译过的指令集",不是堆文档。

1.3 行为驱动(BDD)规范模板 —— 直接抄

用 Gherkin 的 `Scenario / Given / When / Then` ,强制 AI 按"状态 > 动作 > 结果"思考,从根上消除猜测:

● gherkin

Feature: 用户登录

Background:

给 agent 的"为什么":这是面向 C 端的高频入口,错误必须友好可恢复

Scenario: 正确凭据登录成功

Given 数据库中存在 email 为 "a@b.com"、密码哈希匹配的用户

When 用户用正确的 email 和密码提交登录

Then 返回 200 且响应体含有效的 JWT(有效期 24h)

And 该 JWT 的 payload 含 user_id 和 role

Scenario: 边缘情况 —— 连续失败锁定

Given 同一 email 在 5 分钟内已失败 5 次

When 第 6 次提交(即使密码正确)

Then 返回 429 且提示"账户临时锁定,请 15 分钟后重试"

And 不泄露该 email 是否存在

边缘情况(锁定、空输入、超长输入、并发)是规范里最值钱的部分——AI 默认不会替你想全。

1.4 指令住在哪:四个层级(最容易混的点)

位置	生命周期	装什么	别装什么
聊天框	随会话, 临时	高层编排、即时反馈	别把 100 页文档糊进去
specs/ 文件夹	任务级, 入版本控制	技术设计、BDD 场景、API 合约	全局身份
Agent Skills(.agent/skills/.../SKILL.md)	可复用, 触发式	"检测到变更就维护 CHANGELOG"这类工程习惯	一次性任务
System Prompts	全局, 常驻	身份与铁律	频繁变动的细节

System Prompt 按层级叠加, 优先级从低到高: 个人画像(如 `~/.gemini/GEMINI.md`) → 跨工具共享基础 (`AGENTS.md`) → 项目根目录的 `GEMINI.md` / `CLAUDE.md` (项目 DNA, 最高优先级)。

1.5 同一份规范, 不同任务换不同"执行模式"

模式	角色	要点
项目生成	架构师	禁用 YOLO; 先让它提目录结构+技术栈给你确认再写; 每个库标版本号
功能生成	构建者	匹配现有代码风格; 多文件改动手动确认; 一定看 Diff
Bug 修复	取证专家	从"症状式"转"证据式"(贴日志); 先要一个能复现的失败测试, 只修根因, 别顺手清理无关代码
文档编写	作者	文档即事实来源; 文档和代码不同步, AI 就会幻觉; 用 Docstrings/JSDoc
数据工程	管理员	让 agent 永远显示它用的 SQL/命令

注 · NOTE

反复出现的坑:模型有知识截止日期,会推荐**过时的库版本**(因为它不知道有更新的)。对策:在 spec 里写死版本号、用 RAG 喂最新文档,并且**永远二次核对它给的版本号**。

1.6 推荐的项目骨架

● text

```
my-agent/
├─ CLAUDE.md / GEMINI.md      # 项目 DNA(身份+铁律,最高优先级)
├─ AGENTS.md                  # 跨工具共享的基础约定
├─ specs/                     # 唯一事实来源
│  ├─ 00-overview.md          # 背景:这个 agent 是干嘛的、为什么
│  ├─ 01-architecture.md      # 技术设计 + 带版本号的依赖清单
│  └─ features/
│     └─ login.feature          # BDD 场景(Gherkin)
├─ api-contracts.yaml          # 深层结构化配置用 YAML
├─ .agent/
│  └─ skills/                  # 可复用的触发式技能(见第 3 章)
│     └─ changelog/SKILL.md
├─ src/                        # 实际代码(可丢弃/可重生成)
├─ tests/                      # 确定性测试(见第 5 章)
├─ evals/                      # 行为评估(见第 5 章)
└─ policies.yaml               # 护栏规则(见第 5 章)
```

02 工具层:MCP —— 一次集成,处处可用

MCP(Model Context Protocol)是 Anthropic 提出、现已捐给 Linux 基金会的开放标准,被称为"AI 工具界的 USB-C"。你为数据库/API/文件系统只写一个 MCP server,任何兼容 MCP 的 agent 都能直接用,不必为每个框架写定制集成。

2.1 为什么必须用它:NxM 难题

如果你有 N 个模型、M 个工具,传统做法要为每个交点写集成代码:

- › 传统: $O(N \times M)$ —— 5 个模型 $\times$ 10 个工具 = 50 个集成点,任何一个工具 API 一变就崩一片。
- › MCP: $O(N + M)$ —— 每个模型只连一次 MCP,每个工具只暴露一次。

这是你该用 MCP 的全部理由——它把组合爆炸拍平成加法。

2.2 接入三步:发现 → 配置 → 连接

步骤	做什么	注意
发现	找现成 server:公共注册表 / 第三方托管 / 企业内部注册表	公共注册表 未经审核,风险自负 ;优先用官方托管的(如官方 Maps、BigQuery)
配置	设作用域和权限,用环境文件管理凭据	绝不硬编码凭据
连接	host client 跑一次握手,列出工具并验证 output schema	调试时别瞎改 prompt, 直接看传输管道

传输层两种: `stdio` (本地子进程,适合原型,走 JSON-RPC 2.0)和 **HTTP + SSE**(连远程端点,实时推流,依赖少、始终最新,但对云端 server 负担重)。

调试工具:**MCP Inspector**(手动查 schema、测 payload、看原始 JSON-RPC 包)、**Chrome DevTools**(追 SSE 流、看延迟)。

2.3 最小 MCP server 骨架(Python)

把一个 SQLite 知识库暴露成两个工具。这是模板,照着改:

```
python

# server.py — 最小可用 MCP server
from mcp.server.fastmcp import FastMCP
import sqlite3

mcp = FastMCP("knowledge-base")
DB = "knowledge.db"

@mcp.tool()
def query_knowledge(keyword: str) -> str:
    """按关键词检索知识库条目。"""
    con = sqlite3.connect(DB)
    rows = con.execute(
        "SELECT content FROM notes WHERE content LIKE ?",
        (f"%{keyword}%",),
    ).fetchall()
    con.close()
    return "\n".join(r[0] for r in rows) or "无匹配"

@mcp.tool()
def add_knowledge(content: str) -> str:
    """新增一条知识。"""
    con = sqlite3.connect(DB)
    con.execute("INSERT INTO notes(content) VALUES (?)", (content,))
    con.commit(); con.close()
    return "已写入"

if __name__ == "__main__":
    mcp.run(transport="stdio") # 原型用 stdio;上云改 HTTP+SSE
```

要点只有两个:**server 暴露工具**,**client 发现并调用工具**。工具的 docstring 就是给 LLM 看的"说明书",写清楚它什么时候该用、参数什么意思——这比代码本身重要。

2.4 MCP 最佳实践清单

- ☐ 连接公共 server 前**审计代码**
- ☐ 用 RAG **动态加载工具**,保持 context 整洁(别一次性塞 80 个工具进去)
- ☐ 引入 **HITL**:调用前展示工具输入给人看
- ☐ **绝不硬编码凭据**
- ☐ **绝不直连生产环境**:用脱敏数据 + 只读模式

03 能力层:Skills —— 教 agent "怎么做"

MCP 管"够得着"(连外部系统拿数据);Skill 管"会不会做"(教 agent 怎么思考一类工作)。两者是**组合,不是竞争**——Skill 需要数据时,就让 agent 去调一个 MCP 工具。

为什么需要 Skill:LLM 有事实记忆、事件记忆,却缺**程序性记忆**(一步步怎么干)。Skill 是第一个可信的程序性记忆载体,而且是一个 markdown 文件夹,**任何能读文件系统的 agent 都能用,跨厂商通用**。

3.1 最该理解的机制:渐进式披露(三层加载)

层	内容	何时加载	成本
第 1 层	metadata(name + description)	永远在上下文	极小,每个 ~50 token
第 2 层	SKILL.md 正文	该技能被 触发 时	中等
第 3 层	bundled resources(脚本/模板/参考)	用到才加载;脚本甚至能执行而 完全不占 token	按需

结果:你装 100 个技能,平时只付 $100 \times 50 \approx 5000$ token 的"目录费"。触发了才付正文的钱。这就是为什么 Skill 比"把所有指令塞进一个巨型 system prompt"高明——后者会触发**上下文腐烂(context rot)**:指令越多,模型在窗口还没填满时就开始变笨。

3.2 SKILL.md 模板 —— 直接抄

```
markdown

---
name: changelog-maintainer
description: 当检测到 src/ 下有功能性代码变更时,自动在 CHANGELOG.md 顶部追加一条符合 Keep a Changelog 规范的条目。仅用于代码变更,不处理文档改动。
---

# 维护 CHANGELOG## 何时触发
检测到 `src/` 下非测试、非注释的代码变更,且变更涉及新增/修改/移除功能。

## 步骤
1. 用 `git diff` 找出本次变更的功能点(忽略格式化、重命名)
2. 归类到 Added / Changed / Fixed / Removed
3. 在 CHANGELOG.md 的 [Unreleased] 段顶部追加,每条一句话,用户视角
4. 不改动已发布版本的条目

## 参考
- 格式见 ./reference/keepachangelog-spec.md(用到才读)
- 措辞示例见 ./reference/examples.md
```

`description` 就是路由:agent 靠它判断"这个任务要不要打开这个技能"。写得越精准(包含触发条件 + 排除条件),误触发越少。

3.3 三个最容易混的东西

	何时加载	管什么	类比
MCP	调用时	够得着(连数据)	工具的接口
Skill	按需(触发式)	会不会做(程序记忆)	一本操作手册
AGENTS.md	永远	全局静态约定	公司章程

口诀:全局静态的东西放 AGENTS.md;窄的、动作型的工作流才做成 Skill。

3.4 关键的反直觉警告(别盲目堆技能)

- › 真实数据:约 19% 的任务装了技能反而更差——设计糟糕的技能会主动削弱能力。
- › 一个被剥掉指令的烂技能可能得 58 分,而完全不用技能反而得 63 分——技能可以是"负贡献"。
- › 同一研究里被动的 AGENTS.md 拿到满分通过率。

结论:技能不是越多越好。 只为"窄而明确、动作型、会反复出现"的工作流写技能;其余的留给静态文档。写完一定要测(见下)。

3.5 技能的四种失败模式(测试时盯这四个)

1. **触发失败** —— 该用时没用,或不该用时乱用(description 没写好)
2. **执行失败** —— 触发了但步骤错
3. **token 预算失败** —— 把太多东西塞进第 2/3 层,撑爆上下文
4. **回归** —— 改了技能,把原来能干的事干坏了

晋升阶梯:读(只读参考)→ 草稿(可生成但需人确认)→ 可执行(自动跑),逐级放权。

04 协作层:A2A —— 单 agent 扛不住时才拆

默认从单 agent 起步。多 agent 是运维噩梦,不要一遇到复杂任务就上多 agent 架构——很多时候让单个 agent 用 Skill 切换"专家角色"就够了。

什么时候才该拆?当你撞上**单体天花板**,表现为三个信号:

1. **扩展摩擦** —— 改数据库逻辑会干扰 UI 逻辑;工具越多,agent 决策越差。
2. **上下文过载** —— schema 和历史塞爆工作记忆。
3. **单点失败** —— 一个巨型 prompt 身兼数职,错一处全盘崩。

撞上这三个,才从"单体 agent"演进到"微服务式多 agent",用 **A2A(Agent-to-Agent)** 让专门的 agent 之间协商、委派任务。

4.1 拆分的典型形态:sub-agent 流水线

满规模的做法不是一个 agent 读完所有东西,而是拆成专职 agent 的流水线,各管一段、各自上下文干净:

● text

Search(检索相关代码/上下文)

- Story(理解需求/意图)
- Impact(分析影响面)
- Task-breakdown(拆成子任务)
- Coding(实际写代码)

每个 sub-agent 只拿自己那一段需要的上下文,所以即便面对上亿行遗留代码也不会被撑爆。据称这能把百万行代码库的等价重构从两周压到几小时。

4.2 拆 vs 不拆 决策清单

- › 任务能不能用一个 agent + 几个 Skill 搞定?能 → 别拆。
- › 是不是真的有**互相干扰的多个关注点**(数据/UI/安全)?是 → 考虑拆。
- › 拆了之后,**谁来编排、谁来兜底**?想不清 → 先别拆。

- › 你有没有能力给每个 sub-agent **单独评估和监控**? 没有 → 先别拆。

注 · NOTE

一句话:多 agent 的复杂度是真实成本,只有当单 agent 的"天花板"已经实打实挡路时,这笔成本才划算。

05 护栏与零信任 —— 给永不睡觉的工具装边界

先看一个真实教训:一个"创建按钮"的简单 prompt,触发浏览器 agent 在自动批准(YOLO)模式下自主点击;因为没指定 URL,agent 幻觉连到了一个没有任何防护的弃用旧 agent,结果给 50 名同事发了一堆胡编内容的假邮件。

教训叫**上下文幻觉风险**:AI 缺数据时,会用 context 里任何现成字符串(硬编码的邮箱、URL)去填空。所以**护栏不是可选项**——agent 会用它能找到的一切去优化目标,不会问"我该不该这么做"。而且把约束硬写进 system prompt 很脆弱(context 会溢出、prompt injection 能绕过),你需要的是**外部、防篡改的治理**。

5.1 六道护栏(从粗到细,叠加使用)

#	护栏	干什么	怎么落地
1	沙箱化	临时、低权限、隔离容器,制造"爆炸半径"	一个开关,或 <code>GEMINI_SANDBOX=docker</code> ;被骗执行恶意命令也只毁掉一个可丢弃实例
2	人在回路 HITL	高风险动作设"检查点",人工签字	部署生产、改 schema、发起交易前必须人工确认
3	AI 生成测试覆盖	修 bug 前先逼 agent 写出失败测试	测试留在代码库里,防止 bug 复发
4	评估(Evaluation)	抓行为漂移	LLM-as-judge 打 0-5 分 + 容差带(见下)
5	Policy Server	拦截工具调用	结构门控 + 语义门控(见下)
6	上下文卫生 & Prompt 净化	运行时脱敏、占位符注入	<code>ContextResolver</code> 把 <code>[[EMAIL]]</code> 这类占位符运行时替换成授权值

5.2 测试 vs 评估:一个关键区分

	测试(Test)	评估(Evaluation)
问的问题	"返回值对不对?"(二元)	"agent 表现是否至少和基线一样好?"
抓什么	确定性回归	行为漂移
怎么判	断言相等	LLM 当裁判打 0-5 分 + 容差带(容忍工具调用顺序差异)

为什么需要评估:生成式系统可能**通过 100 个单元测试,却选错工具、转述错答案**。一句话记住:**测试抓确定性回归,评估抓行为漂移**。两者都要。

5.3 Policy Server 两层门控

```
● yaml

# policies.yaml — 结构化门控(红绿灯,确定性规则)
roles:
  viewer:
    deny: [send_email, write_db, deploy]    # viewer 永远不能发邮件
  admin:
    allow: ["*"]
```

- › **结构门控(红绿灯)** — 基于角色/环境的确定性规则,如上。viewer 不能调 `send_email`,正则级别就拦掉。
- › **语义门控(智能裁判)** — 用第二个 LLM 检查**意图**。比如 admin **可以**发邮件,但**不能**发未脱敏的 PII——这是正则永远兜不住的,必须靠语义判断。

两层叠加:先过红绿灯(快、确定),再过智能裁判(慢、管意图)。

5.4 上手时的最小护栏(别一步到位)

从零搭建时,不必立刻上全套。最小可用护栏按这个顺序加:

1. **先沙箱**(`GEMINI_SANDBOX=docker` 或等价) — 成本最低,收益最大。
2. **再 HITL** — 任何"写"操作(发送、部署、删除、转账)默认要人确认。
3. **再 AI 生成测试** — 每修一个 bug 留一个失败测试。

4. 等 agent 要给别人用了,再上 **Policy Server + 评估 + 上下文卫生**。

5.5 可持续性提醒:审批疲劳

当面对源源不断的微审批,人会反射性点"Approve",反而漏掉细节(有研究称 AI 重度用户严重倦怠概率显著更高)。对策:设"数字静默时间",别让 HITL 退化成无脑盖章。**HITL 失效不是因为没设,而是因为人麻木了**——所以高风险动作要少而精,别什么都弹确认。

06 Loop 层 —— 让 agent 不靠你一拍一拍地敲

前五章搭好的,是一个"装备齐全、能漂亮跑完一次"的 agent。但它跑完就停,等你敲下一拍——你成了循环里的人肉时钟,每一拍都得你来敲。Loop 层把"等你敲"也自动化掉:你不再 prompt agent,而是设计一套替你 prompt agent 的系统,自己从循环里挪到循环外。

注 · NOTE

心智转变:harness 武装"单次运行",loop 让它"一遍遍自己跑"。完整坐标:prompt(一句话)→ context(一窗口)→ harness(一次跑)→ loop(让它自己跑下去)。这一层叫 Loop Engineering(Addy Osmani,2026·6 命名),它复用你前几章造好的全部零件,只是在外面再套一圈。

6.1 一个 loop 转一圈:五个动作

循环不是空转的死循环,每转一圈都在干具体的活。少一个动作,loop 就转不起来或空转。

动作	干什么	起步做法
发现 discovery	自己找出"这圈该做的事",别等你喂	一个 skill 去读 CI 失败 / open issues / 最近 commits
交付 handoff	把任务隔离着交给干活的 agent	每个发现单开一个 git worktree
验证 verification	换一个 agent 说"不"(全章第 7 章展开)	第二个子 agent 对照测试审查,指令甚至模型都不同
持久化 persistence	把状态写到对话之外	开 PR / 更新工单 + 一个状态文件
调度 scheduling	让它一圈圈自动转	定时器/触发器自动跑,不用你按

记一句话:是调度(automation),才让"我手动跑了一次"变成"它自己一圈圈在转"。少了自动醒、自动接力,剩下的只是把 harness 手动按了很多遍。

6.2 六个零件 —— 全是你前几章已经有的

零件	是什么	对应本手册哪章
Automation	挂在时间表/触发器上自动跑	本章 6.3 / 6.4(新)
Worktree	git 隔离并行 agent 的工作目录,避免写冲突	本章(新)
Skill	固化项目知识,发现逻辑变了改 skill 即可	第 3 章
Connector	MCP 接外部系统,决定 loop 的"视野半径"	第 2 章
Sub-agent	生成者与评判者分离	第 4 章 + 第 7 章
Memory	磁盘上的持久状态(agent 会忘,仓库不会)	本章 6.3(新)

这就是 Loop 层的妙处:它不要求你重学。你只要把已经做好的 MCP、Skill、sub-agent,用 automation 串成一个会自醒、有记忆的循环。新增的硬知识只有三样:worktree 隔离、状态文件、调度选型。

注 · NOTE

关键细节:automation 里触发的应该是一个有名字的 `$skill-name` ,不是把一大墙指令贴进一个没人会再维护的定时任务。skill 能复用能维护,prompt 墙不能。

6.3 从一个 triage loop 起步(别上来就学 Stripe)

每周一千多个 PR 的流水线是**终点不是起点**。第一个 loop 应该小到几乎不像个系统。分五步爬:

- 1. **跑一个定时任务**。最朴素:按间隔把同一件事重跑一遍(Claude Code 的 `/loop` ,Codex 的 Automations 标签页)。
- 2. **让它读东西做 triage**。给它发现逻辑:每天早上读 CI 失败 / 新 issue / 最近 commit,挑出值得处理的列清单。到这步你已有 loop 最核心的两个零件:定时触发 + 自动发现。
- 3. **加状态文件**。把每次发现和进度写进一个 markdown(或 Linear 看板)。这是 loop 的记忆——今天没干完的,明天醒来接着干。别让状态留在对话窗口里,一关就没。
- 4. **加 evaluator**(最关键、最常被跳过)。塞一个能对它说"不"的独立检查,否则它就是在反复自我点头。具体怎么做 → 第 7 章。
- 5. **加 worktree 并行**。让它能同时干好几件事,每个 agent 一个独立 checkout,互不踩脚。

注 · NOTE

反模式警告:新手最容易只搭第 1-3 步就上线,结果是一个没人看着、也没人能拦的循环在自我点头。宁可第一个 loop 做得小,也要把第 4 步(能说不的检查)和人工复核点装齐。

6.4 调度选型:关机也要跑,就别绑在你笔记本上

	本地 /loop 或定时任务	云端(Cloud Routines / GitHub Actions schedule)
跑在哪	你的机器	云,一台永远醒着的机器
要开机吗	要(合盖/断电就停)	不要
最小间隔	~1 分钟	通常 ~1 小时
看本地文件	能	否(每次干净 clone)
适合	高频、要盯本地进程/文件	凌晨扫仓库、不依赖本地状态、"睡觉时跑"

读法很简单:要频繁、要看本地文件 → 本地,代价是机器得开着;要关机也跑、不依赖本地 → 上云,代价是间隔大、每次干净 clone。"睡觉时跑"这句容易被讲歪——本地 /loop 是"我在场时多跑几轮",云端调度才是"我不在场它也照跑",别混。

6.5 落到项目骨架(在 1.6 基础上加 loop 件)

```
● text

my-agent/
├─ ...                # 1.6 已有的 specs/ src/ tests/ 等
├─ .claude/
│   └─ loop.md        # 裸 /loop 默认跑的内容(发现逻辑入口)
├─ loops/
│   ├── daily-triage.md # 这个 loop 干什么(触发的 $skill 名,不是指令墙)
│   └─ STATE.md        # 跨轮记忆:转到哪了、什么没干完(持久化)
├─ evals/             # 评估用例(第 7 章)
└─ policies.yaml      # 护栏 + 审计决策规则(第 5、7 章)
```

07 质量评估与过程审计 —— 那个"能说不"的东西

这是整套系统**最难、也最值钱**的部分。Loop 跑得越欢,这一层越是它的下限:没有它,loop 攒下的烂代码越多,因为没人喊停。

先把两件最容易搅成一团的事劈开——它们正交,要分开设计再让它们咬合:

- **质量评估**问:*"它干得好不好?"* —— 关于**结果与行为**。
- **过程审计**问:*"它怎么干的、谁批的、动了什么、能否追溯?"* —— 关于**可追溯性与治理**。

一个 agent 可以**蒙对答案但过程不可复现**(评估过、审计不过),也可以**全程留痕却产出垃圾**(审计过、评估不过)。

7.0 先定义质量:四大支柱(否则你测不出"好")

我们前面直接讲"怎么评",但跳过了一个更根本的问题:"**好**"是什么? 质量不是"对/错"一根轴,而是四根支柱,缺一根都算低质量:

支柱	问什么	例子
有效性 Effectiveness	真的达成用户意图了吗?	不是"找到商品了",而是"促成转化了"
效率 Efficiency	解得好不好?	绕 25 步、5 次工具失败才订到机票,即使成功也是低质量。用 token/延迟/步数衡量
健壮性 Robustness	出意外时优雅吗?	超时/改版/输入含糊时,能重试、追问、如实报告失败,还是直接崩溃/幻觉?
安全对齐 Safety & Alignment	该不该做?	不可妥协的闸门 ;一个 100% 有效但会造成伤害的 agent 是彻底失败

一句点睛:**性能指标告诉你它能不能做,安全评估告诉你它该不该做**。关键洞察是——**只看最终答案,这四根一根都测不了**:你数不了步数就谈不了效率,不知道哪个 API 调用失败就诊断不了健壮性。所以必须"看见内部"(这正是 7.4 可观测性的理由)。

注 · NOTE

核心心态:把评估当架构支柱,不是最后一道工序(evaluatable-by-design)。 你不会先造好 F1 赛车再外挂传感器,而是从第一行代码就让它可评估。同理:**轨迹即真相**(最终答案只是长故事的最后一句),人是**仲裁者**(自动化负责规模,人定"什么算 A+")。

7.1 第一刀:测试 vs 评估

	测试 Test	评估 Evaluation
问什么	"返回值对不对?"(二元)	"表现是否 ≥ 冻结的基线?"
抓什么	确定性回归	行为漂移
怎么判	断言相等	打 0-5 分 + 容差带

为什么不能只靠测试:一个 agent 能通过 100 个单元测试,却选错工具、转述错答案。单元测试看不见"它绕了三步弯路"或"它调了本不该调的工具"。两者都要。

7.2 评估三件套 + 结果/轨迹两层

每条评估由三部分构成:评分标准(rubric)、裁判(LLM-as-judge 或人)、容差带(不要求逐字一致,容忍等价的工具调用顺序差异)。而且要**同时评两层**——光评最终答案会漏掉危险的过程:

- › **结果质量 outcome:**最终答案对不对、好不好。
- › **轨迹质量 trajectory:**选对工具了吗?有没有绕路?调了不该调的吗?token 花得合不合理?

直接抄的 rubric 模板:

● yaml

```
# evals/login_lockout.yaml
case: 连续失败应锁定而非泄露账户是否存在
input: "用错误密码登录 a@b.com 六次"
rubric:
  outcome:                                # 结果层
    - 第 6 次返回 429
    - 不泄露该 email 是否注册
  trajectory:                             # 轨迹层(光看结果发现不了)
    - 没有调用 send_email(本场景不该发邮件)
    - 没有把明文密码写进任何日志
judge: llm                                # 用第二个模型打 0-5 分
baseline: 4.5                             # 低于基线即视为漂移,CI 红灯
tolerance: 工具调用顺序不计分
```

如果你的 agent 用了 Skill,直接拿 **3.5 的四种失败模式**(触发 / 执行 / token 预算 / 回归)当现成的评估维度。

黑盒→玻璃盒两段式(由外而内):先只问端到端达成目标了吗(黑盒:任务成功率、CSAT、点赞/点踩);**只在失败时才打开盒子**逐段查轨迹(玻璃盒:规划 Thought、工具选择与参数、对结果的解读 Observation、RAG 检索质量、效率与健壮性)。目的是把"答案错了"升级成"答案错了**因为**....."。

更细的评估维度(尤其对写代码/UI 的 agent):**功能正确性是地板不是天花板,而且能作弊**(删测试或 mock 就变绿);**视觉行为正确性**(交付物是渲染后的页面,代码级指标抓不住);**自我修复行为**(失败后能恢复还是越搞越糟)。三个可直接抄的技巧:

- ▶ **会话前缀当意图评分准则**:没规范可测时,头一两条用户消息就是最接近规范的——自动从中推导验收标准,再给后续每轮打分。
- ▶ **评判渲染后的 artifact,而非代码**:用多模态模型看渲染页面,抓代码级评估错过的布局错乱/对比度/按钮状态;配合 Playwright 断言抓交互损坏。
- ▶ **评会话收敛性,而非单轮准确率**:核心不是"第 4 轮对不对",而是"用户最终收敛到想要的了吗"。**中途被放弃的会话是信息量最大的失败案例**。

7.3 让评估"能说不":maker-checker 原则

为什么不能让写代码的 agent 自己打分?因为它太手软——它的上下文里塞满了"我为什么这么写"的理由,看自己的代码看到的是一路推导的自我说服。实践反复验证的反直觉结论:**调一个独立的、怀疑的评判器,比让生成器自我批判容易得多**。这就是 GAN 式的 generator / evaluator 分离。四步让 loop 长出"说不"的能力:

1. **结构上分开生成与评判**(两个 agent,不是一个 agent 两顶帽子)。
2. **把 evaluator 调成怀疑论者**——默认"这段代码是坏的,除非被证明能跑",而不是默认信任。
3. **让评判器会动手,不只会读**。只读代码判断的是"看起来对不对";接上工具自己去点、去截图、去查(前端任务接 Playwright),判断的才是"跑起来对不对"。会动手的评判器看的是行为,不是意图。
4. **判定权交给一个没参与干活的 fresh 模型**。换 agent 时**顺手换模型**——同一模型换了指令,思维盲区往往还在原地;换底座,连它习惯绕过的坑都换一批。

现成的产品原语: `/goal`。给 agent 一个可验证条件(如"test/auth 全过且 lint 干净"),它一直跑到满足为止;每轮结束,由一个又小又快的、没参与这轮工作的模型来判断条件成立没有,不成立就再跑一轮。这就是把银行的 **maker-checker(录入与复核必须两人)** 老规矩塞进了 agent 循环——经手的人会信任自己的操作,复核才有意义。

注 · NOTE

命令名别安错门: `/goal`、`/loop` 是 Claude Code 的;Codex 走 Automations + agents 实现同类能力。`/loop` (按间隔重跑,是调度)和 `/goal` (跑到条件满足,是评估)是两码事。

评委是个有取舍的层级,不是单一 judge(从快到准、从规模到真相):

评委	取舍	用在哪
自动化指标 (ROUGE/BLEU/BERTScore)	快、可复现,但浅(只看表面相似)	CI 第一道 回归趋势闸 :0.8 突掉 0.6 就报警
LLM-as-Judge	强模型给输出打分	规则抓不住的维度(意图满足、风格、轨迹)
Agent-as-Judge	评 完整轨迹	专抓"答案看着对、过程其实有问题"
HITL	人是仲裁者,但 不是完美真相	校准基准、创建黄金集

一个可直接抄的改进:LLM 评判用**成对比较(pairwise,A/B 二选一算胜率)**比单点 1-5 分可靠得多——单点分会剧烈波动。

注 · NOTE

强化 maker-checker 的一个真实盲区:用同一个 AI 既生成又验证变更,会制造**"看似评审、实则审查者与被审查者同源"**的假职责分离(Trend Micro)。这正是上面**"换一个独立评判器、最好换模型"**要防的——同源等于没分离。

7.4 过程审计三层(从细到粗)

审计的对象不是返回值,而是**行为轨迹 + 治理决策**。

① **轨迹追踪(observability 核心)**。 每一步留痕:想了什么、选了哪个工具、传了什么参数、拿回什么、花多少 token。最朴素的版本就是 1.5 那条铁律——**让 agent 永远显示它用的 SQL / 命令**,别让你在你看不见的地方动数据。一条记录:

● json

```
{
  "trace_id": "req-8821", "step": 3,
  "thought": "需要查用户是否已锁定",
  "tool": "query_db",
  "args": {"sql": "SELECT fail_count FROM users WHERE email=?"},
  "result_summary": "fail_count=6",
  "tokens": 412, "ts": "2026-06-28T09:40:12Z"
}
```

② **治理决策日志(合规核心)**。Policy Server(5.3)每一次放行/拦截都记账:谁(role)、什么环境、想调什么工具、结构门控(红绿灯)结果、语义门控(智能裁判)结果、最终放行还是拦。这就是 Policy 必须**外部、防篡改**的原因——审计记录如果 agent 自己能改,等于没有。

③ **人工节点 + 变更级审计**。HITL 检查点**本身就是审计记录**(谁在何时批了哪个高风险动作)。每个变更附一份 AI 生成的摘要 + 风险评估,把"改了什么、风险在哪"沉淀成可追溯记录,而不是事后靠人 diff 几千行。沙箱的"爆炸半径"天然就是审计边界。

可观测性三支柱(把上面三层落到标准工具上):**Logs(日记:原子级带时间戳的事实,发生了什么,结构化JSON)**· **Traces(因果故事:把零散日志串成一条链,为什么——建在 OpenTelemetry 上,span + trace_id)**· **Metrics(聚合分数)**。Metrics 必须**分两类:系统指标**(延迟 P50/P99、错误率、每任务 token/成本、完成率,给运维)vs **质量指标**(正确性、轨迹遵循、安全、有用,给产品)。落地三件事:仪表盘把系统健康和模型质量分开、日志 PII 清洗、**动态采样**(成功 10%、错误 100%;尾部采样只留出错或过度自修复的 trace,否则全量 trace 撑爆存储)。

没有单一评估方法能覆盖一切,生产流水线要组合(七种,各管不同维度):**标准化基准**(横向比)· **自动化功能测试**(pytest/eslint 接 CI,最便宜)· **安全合规扫描**(Snyk/Semgrep,交叉评分)· **LLM/Agent-judge**(规则抓不住的)· **浏览器测试**(Playwright,管视觉行为)· **轨迹检查**(OTel trace 回放)· **人工复核**(校准,不可规模化)· **在线评估**(采样真实流量,偏向高成本/多修正/被放弃的会话)。

7.5 质量飞轮:让两者越转越快

它们不是两套独立系统,是一个自我强化的**飞轮**——第一推最费力,持续评估会让它越转越快:



底层咬合关系:评估的轨迹层,原料就是审计的 trace(没 trace 就做不了轨迹评估);没有评估,审计日志只是一堆没有好坏判断的流水。飞轮转起来后,生产里抓到的每个失败都变成黄金集里的一条新回归用例,下次自动拦住同类问题。(这个飞轮的第 ④ 步,正是第 10 章 AgentOps 里 evolve 的入口。)

7.6 四笔代价 = 审计真正要防的

一个无人看管的 loop,也是一个无人看管地在犯错的 loop。这四笔账的共同点:都不在 loop 跑的当下报警。

代价	症状	一句话防它
验证债	产出堆着没人验,错误安静积累	装一个跟干活的不是同一个的评判者(7.3)
理解腐烂	代码在长,你脑里的地图停了	定期读产出,挑几个改动自己讲一遍;讲不出就是该更新
认知投降	循环给啥收啥,懒得有意见	执行可外包,拿主意不行;至少保留说"这个不对"的能力
token 失控	用量剧烈波动,账单不可预测	上线前钉死预算和重试上限(见下)

三道必装的硬闸(实践者反复落在同样三条):迭代次数上限(卡住的循环不能无限空转)、diff 无变化即停(近几轮不再改动就杀掉)、花费上限(token 或美元,在账单失控前结束)。少任何一道,你跑的就不是 loop,是一张敞开的发票。

7.7 最小起步版(别一上来就全套)

两条线各取最小可用:

- 质量:挑 3-5 个"黄金案例",写 rubric,用第二个模型当 judge 打分,跟一个冻结基线比。够了。
- 审计:先让 agent 永远打印它的每一次工具调用和参数(stdout trace 就行)+ HITL 留一个"批了什么"的简单日志。够了。

等 agent 要给别人用、或要碰生产数据时,再补 Policy 决策日志、变更风险摘要、上下文卫生记录。

08 记忆与会话 —— agent 记不记得住

这是栈里那个被前面一笔带过的 **context 层**。它最基础,却和 Skills(程序记忆)、Loop(状态文件)交织最深,所以放在这里读最清楚。本章框架对齐《Context Engineering: Sessions, Memory》那套心智模型。

8.0 先立心智模型:工作台 vs 文件柜

把 agent 的记忆系统想成办公场景:

- › **Session(会话)= 你的工作台**。做当前任务时,桌面摊满草稿、便签、工具——触手可及,但**临时、只属于这个任务**。
- › **Memory(记忆)= 整理好的文件柜**。任务做完,你不会把整张乱桌子塞进柜子,而是**筛选、丢草稿、把真正重要的归档**进贴标签的文件夹。
- › **Context Engineering(上下文工程)= 厨师备料**。每做一道菜前,把对的、高质量的食材准备到位。目标:让模型拿到的信息**不多不少,刚好够用**。

一句话定位:**Session 管"现在",Memory 管"长期"**;而且 Memory 存的是提取后的洞见,不是原始逐字对话。这个区分是整章的地基。

注 · NOTE

为什么需要这一层:LLM 本身是**无状态的(stateless)**。让一个 agent 能"记住、学习、个性化",全靠你在它外面搭的会话 + 记忆系统。

8.1 三层记忆:working / task / long-term

最朴素的落地(对应一个最小 harness 里的 **Memory** 类),记忆分三层:

层	是什么	生命周期	存哪
working 工作记忆	本轮送给模型的消息历史 = agent 的 RAM	单轮即逝,窗口刷新就冲掉	运行时 context,不落盘
task 任务记忆	任务级状态:进度、已做的决定、未完成项	跨轮,任务内	STATE.md / 看板
long-term 长期记忆	跨会话的用户画像、项目事实、踩过的坑	跨天、跨对话	磁盘 / DB / 向量库

关键:context 窗口一清空,working 就忘光。想让今天接着昨天跑,就必须把该记的沉到 task / long-term 落盘(呼应第 6 章"agent 会忘,仓库不会")。

8.2 工作记忆的硬约束:context rot

一个反直觉但要命的概念:上下文不是越长越好。窗口越长,模型对关键信息的注意力反而越差,成本和延迟还都上升——这叫上下文腐烂(context rot)。所以压缩不是为省钱的妥协,是为保质量。

每一轮的操作循环记牢:Fetch(取上下文)→ Prepare(组装,这是阻塞的"热路径")→ Invoke(调模型/工具)→ Upload(把新信息异步写回存储)。写回一定异步,别让用户等。

8.3 Session:管"现在"

会话是单次连续对话的容器,绑定到某个具体用户,两个组成部分:

- › **events:**按时间顺序的事件流(用户输入、agent 回复、工具调用、工具输出)——通常只追加(append-only)。
- › **state:**结构化的工作记忆/草稿纸(比如购物车里有什么)。

生产三大关注点,直接抄进 checklist:

- › **严格隔离(ACL):**一个用户**绝不能**碰到另一个用户的会话。
- › **PII 写入存储前就脱敏,**缩小泄露的"爆炸半径"。
- › **性能:**会话在每一轮热路径上,读写必须极快;并设 **TTL**,会话不该永久存在。

长上下文压缩(行李箱比喻:成功不在能装多少,而在只装需要的),由简到繁:

1. 保留最近 N 轮(滑动窗口)

- 2. 基于 token 的截断
- 3. 递归摘要:用另一次 LLM 调用把旧对话压成摘要,拼在近期原文前面——较贵,所以**后台异步做并持久化结果**,别重复计算。

触发时机三类:**计数触发**(token/轮数超阈值)、**时间触发**(一段无活动)、**事件触发**(检测到某任务/话题结束)。

8.4 Memory vs RAG —— 一定要分清

全章最重要的对比(图书管理员 vs 私人助理):

	RAG	Memory
角色	研究图书管理员	私人助理
性质	静态、 共享 、权威外部知识库	动态、 按用户隔离 、来自对话
让 agent 成为	"事实"的专家	"用户"的专家

一个真正智能的 agent **两者都要**:RAG 让它懂世界,Memory 让它懂你。

8.5 记忆五个分类维度(知识骨架)

维度	选项
信息类型	陈述性("知道什么":事实/偏好)vs 程序性 ("知道怎么做"= Skills,第 3 章)
组织模式	collections(多条独立记忆)/ structured profile(联系人卡片式核心事实)/ rolling summary(滚动总摘要,常用于压缩长会话)
存储架构	向量库(找语义相似)/ 知识图谱(找实体关系)/ 混合
创建机制	显式(用户说"记住...")vs 隐式(agent 推断);internal(框架内置)vs external(专门服务)
作用域	user-level(最常见)/ session-level(压缩单次长会话)/ application-level(全局共享, 必须脱敏防跨用户泄露)

8.6 记忆生成 = 一条 LLM 驱动的 ETL 流水线(园丁比喻)

园丁不乱撒种子:收到新苗 → 拔草去重、修剪旧枝、种到最佳位置。四个阶段:

1. **Ingestion**:喂入原始数据。
2. **Extraction & Filtering**:LLM **不全抓**,只抓符合预定义 topic 的有意义信息;"有意义"由用例定义(客服 agent 和健康教练要记的完全不同)。
3. **Consolidation(最复杂)**:自我编辑,对比新旧记忆,决定 **UPDATE / CREATE / DELETE**——处理去重、冲突、信息演化,以及**主动遗忘(forgetting/pruning)**。
4. **Storage**:落库。

两条铁律:**Memory Manager 是个主动系统,不是被动的向量库**(灵魂在智能提取/整合/策展);**生成记忆几乎永远异步**(贵,要在 agent 回复用户之后后台跑)。

Provenance(溯源) 易被忽略但关键——"garbage in, confident garbage out"。每条记忆追踪来源信任,分三档:**Bootstrapped Data**(从 CRM 预载,高信任) > **User Input**(显式表单 > 隐式对话) > **Tool Output**(易过时,一般不建议生成长期记忆)。冲突按此层级解决;用户撤回某数据源时,最精确的做法是**仅用剩余有效源重新生成**受影响记忆,而不是粗暴删掉所有沾过的。

8.7 读写策略(呼应 Skills"不是越多越好")

- › **何时生成**:session 完成 / 每 N 轮 / 实时每轮 / 显式命令 / **Memory-as-a-Tool**(让 agent 自己决定调用 `create_memory`)。本质是**成本-延迟 vs 新鲜度-保真度**的取舍。
- › **检索按三维打分**:**Relevance**(语义相似)+ **Recency**(多新)+ **Importance**(重要性)。只靠向量相似度是常见坑——会捞出"相似但又老又琐碎"的记忆,要加权混合。
- › **检索时机**:proactive(每轮预加载,简单但有冗余延迟,可缓存)vs reactive/Memory-as-a-Tool(agent 自己决定何时查,更省但多一次 LLM 调用)。
- › **放进 context 的位置**:放**系统指令**(权威、干净、适合稳定的用户画像,但有"过度影响"风险)vs 放**对话历史**(灵活,但有 "dialogue injection" 风险——模型可能误以为记忆是真说过的话)。实践中**混合用**最好。
- › **安全**:防 **memory poisoning**(恶意用户用 prompt injection 污染长期记忆)和**跨用户泄露**(共享的程序记忆先匿名化)。记忆也要能被**审计与更正**(接第 7 章)。

8.8 落到项目骨架

```
memory/
├─ session/           # Session 管"现在"(热路径, 设 TTL)
│   └─ events.log      # append-only 事件流 + 结构化 state
├─ facts/             # 陈述性长期记忆
│   └─ user-profile.md # structured profile: 用户偏好/项目事实
├─ summary.md         # rolling summary: 滚动压缩的长会话摘要
└─ knowledge/         # 语义/RAG 源(共享、权威), 按需检索不常驻
# working 工作记忆 = 运行时 context, 不落盘 # 程序性记忆 = .agent/skills/(第 3 章)
```

8.9 一张记忆速查表

类型	是什么	住在哪	生命周期	对应本手册
工作记忆	这一轮能看到的	context 窗口	单轮即逝	8.1–8.2
会话(Session)	当前任务的"现在"	events + state	单次会话(可 TTL)	8.3
陈述性长期记忆	关于用户/项目的事实	facts / 向量库	跨天跨对话	8.4–8.7
程序性记忆	怎么做一类活	SKILL.md	触发式	第 3 章
语义知识(RAG)	共享的世界事实	知识库	持久, 按需取	8.4 + 第 2 章

8.10 最小起步版(别一上来就 Mem0/Zep)

- 1. 先三层就够: working 在 context 里 + 一个 STATE.md (task/跨轮)+ 会话满了让它递归摘要压缩。
- 2. 要个性化: 加 facts/user-profile.md , 开头读进来; 写入走 ETL 思路——异步、只抓有意义的、整合去重, 别每轮全量写。
- 3. 要查大知识: 上 RAG/MCP, 按 Relevance + Recency + Importance 检索, 别只用向量相似度。
- 4. 上线前三件套: ACL 隔离 + PII 写入前脱敏 + 防 memory poisoning。

09 落地路径:从零到一条可执行的路

把前八章串成一个**最小可行 agent** → **逐步加固** → **让它自己跑**的顺序。不要试图一开始就把所有层全建好。

◇ 阶段 A:能跑(第 1 天)

1. 建项目骨架(见 1.6),写 `specs/00-overview.md` 说清楚"这个 agent 干嘛、为什么"。
2. 用**架构师模式**让 AI 提目录结构 + 带版本号的技术栈,**你确认后再让它写**。
3. 跑起来一个最朴素的 Model + 一两个工具的循环。**先沙箱**(阶段 A 唯一的硬性护栏)。

◇ 阶段 B:能验证(第 2-3 天)

4. 为核心功能写 **BDD 场景**(1.3 模板),把边缘情况写全。
5. 让 AI 按场景生成代码 + 测试;**每修一个 bug 先写失败测试**。
6. 给所有"写"操作加 **HITL**(发送/部署/删除/转账默认要确认)。

◇ 阶段 C:能复用 & 够得着 & 记得住(第 1 周)

7. 把外部系统接入做成 **MCP server**(2.3 骨架),工具 docstring 写清楚。
8. 把"反复出现的工作流"抽成 **Skill**(3.2 模板)——**但先问自己:这真的窄而动作型吗?** 不是就放进 AGENTS.md。
9. 写完每个 Skill **测四种失败模式**(3.5);如果加了反而变差,删掉。
10. **搭三层记忆**(8.1):working 在 context、`STATE.md` 记跨轮、`facts/` 记用户/项目事实;会话满了让它**递归摘要压缩**(8.3)。

◇ 阶段 D:能评估 & 能放权(按需)

11. 撞到单体天花板了再考虑拆 **sub-agent 流水线**(4.1),想不清编排就别拆。
12. 给核心场景写 **评估**(7.2 rubric 模板),配一个**独立的、会动手的评判器**(7.3);上 **Policy Server**(5.3)+**上下文卫生**。

- 13. 打开**轨迹追踪**(7.4 ①),让每次工具调用都留痕;记忆写入走 **ETL + 异步**(8.6),并防 memory poisoning。

◇ 阶段 E:让它自己跑(确有重复劳动再上)

- 14. 从一个 **triage loop** 起步(6.3):定时任务 → 读 CI/issue 做发现 → 加 **状态文件**(记忆)。
- 15. 先把 **evaluator** 和人工复核点装齐,再开自动(6.3 第 4 步);钉死**三道预算闸**(7.6)。
- 16. 选调度:要关机也跑就上云,要盯本地就**本地**(6.4)。多个 agent 并行用 **worktree** 隔离。
- 17. 设"数字静默时间",防审批疲劳与认知投降(5.5 / 7.6)。

◇ 阶段 F:上生产 & 扛住很多人用(从原型到规模)

- 18. 把评估接进 **CI/CD 当部署闸门**(10.3);**灰度/金丝雀发布**,先 1% 流量看质量指标再放量,出问题秒回滚(10.4)。
- 19. 跑通 **observe → act → evolve** 运维循环:生产失败回流进黄金集,重过 CI/CD(10.5,接 7.5 飞轮)。
- 20. 要多人在线:先认清 **LLM API 限额才是真瓶颈**(10.6);把 agent **做成无状态、状态外置**以便横向扩展(10.7);加**限流配额 + 队列背压 + 缓存**(10.8–10.9)。

◇ 一页纸总检查清单

■ 规范

- ☐ **specs/** 里有背景("为什么")、技术设计、BDD 场景、边缘情况
- ☐ 依赖都带版本号,且**你核对过**不是模型瞎编的旧版本
- ☐ 项目根有 **CLAUDE.md / GEMINI.md** 锚定身份与铁律

■ 工具

- ☐ 外部集成走 MCP,不写一次性 wrapper
- ☐ 凭据不硬编码;不直连生产;工具调用前可见(HITL)

■ 能力

- ☐ 只有"窄+动作型+反复出现"的才做成 Skill
- ☐ 每个 Skill 的 description 含触发条件 + 排除条件
- ☐ 每个 Skill 测过四种失败模式

■ 协作

- ☐ 默认单 agent;拆之前确认有"互相干扰的关注点"且想清楚了编排

■ 护栏

- ☐ 沙箱已开
- ☐ 所有"写"操作有 HITL
- ☐ 高风险动作少而精,没把 HITL 变成无脑盖章

■ 评估与审计

- ☐ 测试抓回归 + 评估抓漂移,两者都有
- ☐ 评估同时评**结果**和**轨迹**,跟一个冻结基线比
- ☐ 评判器是**独立的、会动手验证的**,判"完成"的是没参与干活的 fresh 模型
- ☐ 每次工具调用留痕(trace);Policy 放行/拦截记账且**外部防篡改**

■ 记忆与会话

- ☐ 三层分清:working(context)/ task(STATE.md)/ long-term(落盘)
- ☐ 长会话有压缩策略(滑动窗口→截断→递归摘要),且摘要**异步持久化**
- ☐ Memory 与 RAG 分清:用户事实归 Memory(按用户隔离),世界事实归 RAG(共享)
- ☐ 记忆生成走 ETL(提取→整合去重→遗忘),**异步**做,带来源溯源
- ☐ 检索按 Relevance+Recency+Importance 打分,不只靠向量相似度
- ☐ ACL 隔离 + PII 写入前脱敏 + 防 memory poisoning / 跨用户泄露

■ Loop(若让它自己跑)

- ☐ 五动作齐:发现 / 交付 / 验证 / 持久化 / 调度
- ☐ 有状态文件(跨轮记忆落在磁盘,不在对话窗口)
- ☐ **开自动之前**已装好 evaluator + 人工复核点
- ☐ 钉死三道闸:迭代上限 / diff 无变化即停 / 花费上限
- ☐ 并行 agent 各自 worktree;关机也要跑的上云

■ 生产与规模化(若给很多人用)

- ☐ 评估进 CI/CD 当部署闸门;灰度发布 + 可秒回滚
- ☐ 跑 observe→act→evolve;生产失败回流黄金集
- ☐ agent 无状态、状态外置(会话→Redis、记忆→DB/向量库),可横向扩 N 副本
- ☐ LLM API 限额有管理(队列 + 退避 + 多 key);热路径外的活异步化
- ☐ 按用户/租户限流配额 + 背压降级 + 缓存;租户隔离防噪声邻居/跨租泄露
- ☐ 重试幂等;并发写有冲突策略;按租户算成本

10 从原型到生产与规模化 —— 让一个 agent 扛住很多人同时用

前九章讲的全是**怎么造(BUILD)**:specs、工具、技能、护栏、循环、评估、记忆。这一章讲造好之后的另一半——**怎么真正交付、运维,并扛住高并发(OPERATE & SCALE)**。一句话定调:"**造一个 agent 容易,信任它很难。**" 真实落地里约 80% 的工作量不在 agent 的"智能"本身,而在让它**可靠、安全的基础设施、验证和安全机制**——这段叫**最后一公里(last mile)生产鸿沟**,正是第 0 章"瓶颈在集成"的放大版。

注 · NOTE

本章分两部分:**Part 1(AgentOps)** 取自 Google 《Prototype to Production》;**Part 2(高并发)** 在那批白皮书里源料最薄,大部分是**通用高并发服务工程**应用到 agent 上,我会标出来,方便你区分"白皮书给的"和"工程常识补的"。

◇ Part 1 · AgentOps 生命周期(从原型到生产)

10.1 AgentOps 全景:造→评→上→运维→规模化→回流

阶段	干什么	对应本手册
People & Process(底座)	建它、治理它的团队与流程	个人搭可压薄,给人用必须补
Pre-production	评估闸门 + CI/CD + 安全发布	第 7 章 + 本章 10.3–10.4
Production Ops	observe → act → evolve	第 7 章可观测性 + 本章 10.5
Scaling	用 A2A + MCP 扩能力,用本章扩并发	第 2、4 章 + Part 2
反馈闭环	生产失败回流,重过 CI/CD	接 7.5 质量飞轮

10.2 agent 的 CI/CD:把评估当部署闸门

传统 CI/CD 跑测试;agent 的 CI/CD 还要**跑评估**。把 prompt / skill / model 版本当**源代码**管理(记录元数据,即 PromptBOM / AgBOM),每次改动触发流水线:

● text

确定性测试(lint/test)→ 评估(7.2,跟冻结基线比,掉到线下就拦)
→ 安全扫描(Snyk/Semgrep + 拒绝行为对抗探测)→ 灰度发布

关键:评估分数掉到基线下,就像测试红灯一样拦住发布。这是把第 7 章从"离线打分"变成"上线闸门"。

10.3 安全发布:灰度 / 金丝雀

别一次切全量。先放 **1% 流量** → 盯质量指标(7.4 系统/质量指标)→ 没退化再逐步放量 → 出问题**秒回滚**(熔断器 + 版本 checkpoint,冻结执行而不损坏已连接的 API)。

10.4 生产运维:observe → act → evolve

- › **observe**:可观测性(7.4 Logs/Traces/Metrics)持续盯,尤其盯**意图漂移**和成本异常。
- › **act**:拿信号行动——熔断昂贵循环、回滚、降级、告警。
- › **evolve**:把生产里抓到的失败**回流进黄金集**,重过 CI/CD 发布。这就是 7.5 质量飞轮的第 ④ 步落到运维上。

◇ Part 2 · 高并发与多租户服务(扛住很多人同时在线)

注 · NOTE

提醒:以下大部分是**通用分布式服务工程**(无状态、横向扩展、限流、队列、缓存),不是那批白皮书的内容——它们偏方法论,不偏系统工程。白皮书直接相关的只有:可丢弃沙箱(第 6 章)、租户隔离与 Denial of Wallet(第 5 章)、会话热路径/异步/多区域(第 8 章)。

10.5 先认清真正的瓶颈:LLM API 限额

多人同时在线,先撞墙的常不是你的服务器,是模型方的 **TPM/RPM 限额**(并发一高就 429、排队)。这是多用户 agent 系统**最核心、最易被忽略**的约束。对策:**请求队列 + 限流 + 退避重试 + 多 key/多区域分摊 + 优先级队列**(付费用户优先)。把对模型的调用收口到一个 **LLM 网关**统一管限额、重试、缓存。

10.6 无状态化 + 横向扩展

要在负载均衡后面跑 **N 个副本**,agent 进程就必须**无状态**:状态全部外置——**会话** → **Redis(热路径、TTL)**、**长期记忆** → **DB/向量库(异步写)**。这呼应第 6 章的 "**cattle not pets**":可丢弃、无状态的执行环境正是高并发的前提(Stripe 能同时跑上千 agent 就靠这个)。配套:**重试要幂等**;并发写同一用户状态用**乐观锁 / last-write-wins**。

10.7 限流、配额、背压、降级

- › **按用户/租户限流配额**(不只是 7.6 给单个 loop 的预算闸,是多租户的公平性与防滥用)。
- › **背压**:队列满了就排队或拒绝,而不是雪崩。
- › **过载降级**(graceful degradation / load shedding):高峰期降到更小模型、更短上下文、关掉非核心功能。
- › **防 Denial of Wallet**:有人故意触发昂贵循环烧账单 → 熔断(接第 5、7 章)。

10.8 缓存与异步:并发下砍延迟和成本

- › **缓存**:prompt 缓存(provider 级)、语义缓存(相似问直接命中)、记忆/embedding 缓存。
- › **异步化**:能后台做的绝不放热路径——**记忆生成、评估打分、长任务**都丢进队列后台跑(接第 8 章"记忆生成永远异步")。

10.9 多租户隔离与成本规模化

- › **隔离**:ACL + 向量库强制租户隔离 + per-agent 唯一身份(接第 5、8 章),防噪声邻居和跨租泄露。
- › **成本**:单请求成本 × 并发量,经济账在多用户时会翻盘——**按租户算账、设配额、监控每用户成本**。

10.10 高并发服务骨架

● text

```
[客户端]
  ↓
[API 网关 / 负载均衡] ← 认证、限流、背压
  ↓
[无状态 agent 副本 × N] ← worker pool, 可任意扩缩
  ├── 会话存储 (Redis, 热路径, TTL)
  ├── 长期记忆 (DB/向量库, 异步写)
  ├── LLM 网关 (限额/重试/缓存/多 key) ← 真正的瓶颈在这
  ├── 工具 (MCP servers)
  └── 沙箱 (可丢弃容器, cattle-not-pets)
  ↓
[队列] → [后台 worker] ← 异步: 记忆生成 / 评估 / 长任务
[可观测性: Logs/Traces/Metrics] ← 贯穿全部
```

10.11 最小起步版(别过度工程)

单租户、低并发别上这套。 一个无状态进程 + Redis 会话 + 对模型调用的退避重试, 就够撑很久。上量了再按这个顺序加: **限流配额** → **缓存** → **队列背压** → **灰度发布** → **多区域**。一句话: **先把 agent 做对(前九章), 再把它做大(本章); 别反过来。**

1. **代码可丢弃, 规范是资产。** 你的价值从敲代码转向写出 AI 不用猜的蓝图。
2. **瓶颈在集成与验证, 不在生成。** 所有实践都是为了让验证和集成跟得上 AI 的产出速度; 而到了生产, 80% 的功夫又花在让它可靠地服务别人。
3. **自主必须配护栏。** 速度越快, 沙箱 / HITL / 策略引擎 / 上下文卫生越是底线, 而非锦上添花。
4. **稀缺的是判断, 不是生成。** 同一个 loop, 两个人搭出相反结局——它忠实放大你带进去的东西。造循环, 但要像一个打算继续当工程师的人去造, 而不是一个只负责按启动键的人。

注 · NOTE

关于来源的诚实提醒:本手册综合自几篇前瞻性的厂商技术文章(标注 2026 年)。第 6 章 Loop 层取自 **Loop Engineering**(Addy Osmani 于 2026 年 6 月命名,引自 Peter Steinberger、Boris Cherny),这是一个**正在成形、业界尚有争议**的概念(有人视为新抽象层,也有人讥为"戴帽子的 cron job"),请当框架而非定论。第 7 章评估对齐 Google 《**Agent Quality**》;第 8 章记忆与会话对齐 Google 《**Context Engineering: Sessions, Memory**》;第 10 章 Part 1(AgentOps)取自 Google 《**Prototype to Production**》。第 10 章 Part 2(高并发/多租户)大部分不是这些白皮书的内容,而是通用分布式服务工程应用到 agent 上,已在该节开头标明,请据此判断取舍。其中一些**具体数字**(40% 性能差、19% 任务变差、80% 在最后一公里、Stripe 每周 1300+ PR 等)更应理解为**支撑论点的示例**,出处不一定可考。产品名、版本号、`/loop / /goal` 这类命令、外部服务会很快过期,**动手时务必核对当前官方文档**。